

# Prédiction de la consommation de carburant avec un LSTM compact (TinyML)

Rapport du projet de fin de module

Par : Abdelhamid SAIDI  
Fichier source : `fuel-lstm.ipynb`

module : Deep Learning

## 1 Contexte et objectif

Ce projet vise à prédire la consommation de carburant (L/100 km) d'un véhicule à partir de variables physiques et d'émissions. L'objectif secondaire est d'obtenir un modèle très compact pouvant être converti pour une exécution TinyML sur microcontrôleur.

Le notebook implémente une chaîne complète : chargement des données, nettoyage, EDA, préparation, architecture LSTM compacte, entraînement, évaluation et export TinyML (optionnel).

## 2 Données utilisées

### 2.1 Source et format

Le jeu utilisé est 'Vehicle Attributes and Emissions' (Kaggle : krupadharamshi/fuelconsumption). Le fichier principal attendu est `FuelConsumption.csv`.

### 2.2 Variables retenues

Le notebook utilise quatre variables numériques principales :

- `ENGINE SIZE`;
- `CYLINDERS`;
- `COEMISSIONS`;
- `FUEL CONSUMPTION` (cible).

La présentation est multivariée : les trois premières servent d'entrées et la dernière est la variable cible continue.

## 3 Prétraitement et ingénierie des variables

### 3.1 Validation et nettoyage

Le notebook supprime les lignes manquantes et les doublons avec : `lecture`, `dropna()` et `drop_duplicates()`. Les colonnes sont nettoyées des espaces dans leurs noms.

### 3.2 Standardisation

Les variables d'entrée sont standardisées via `StandardScaler` ajusté uniquement sur l'ensemble d'entraînement, garantissant l'absence de fuite d'information.

### 3.3 Mise en forme pour LSTM

Les vecteurs d'entrée sont remodelés en forme (`samples`, `timesteps=3`, `features=1`) comme attendu par l'architecture LSTM employée.

## 4 Analyse exploratoire

Le notebook produit plusieurs figures sauvegardées dans le dossier `figs` :

- `pairplot.png` — pairplot des variables numériques ;
- `heatmap.png` — matrice de corrélation Spearman.

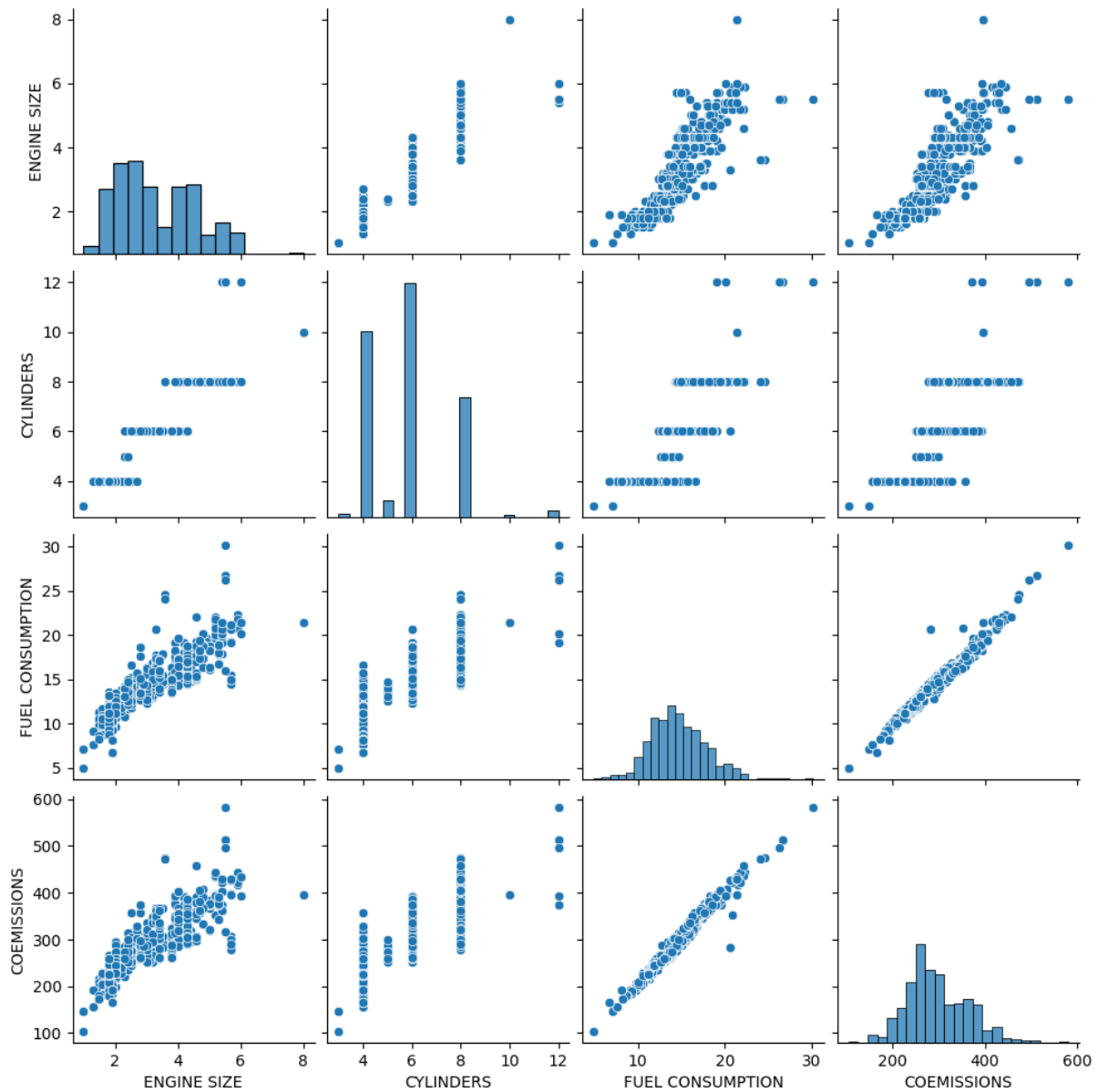


FIGURE 1 – Pairplot des variables numériques

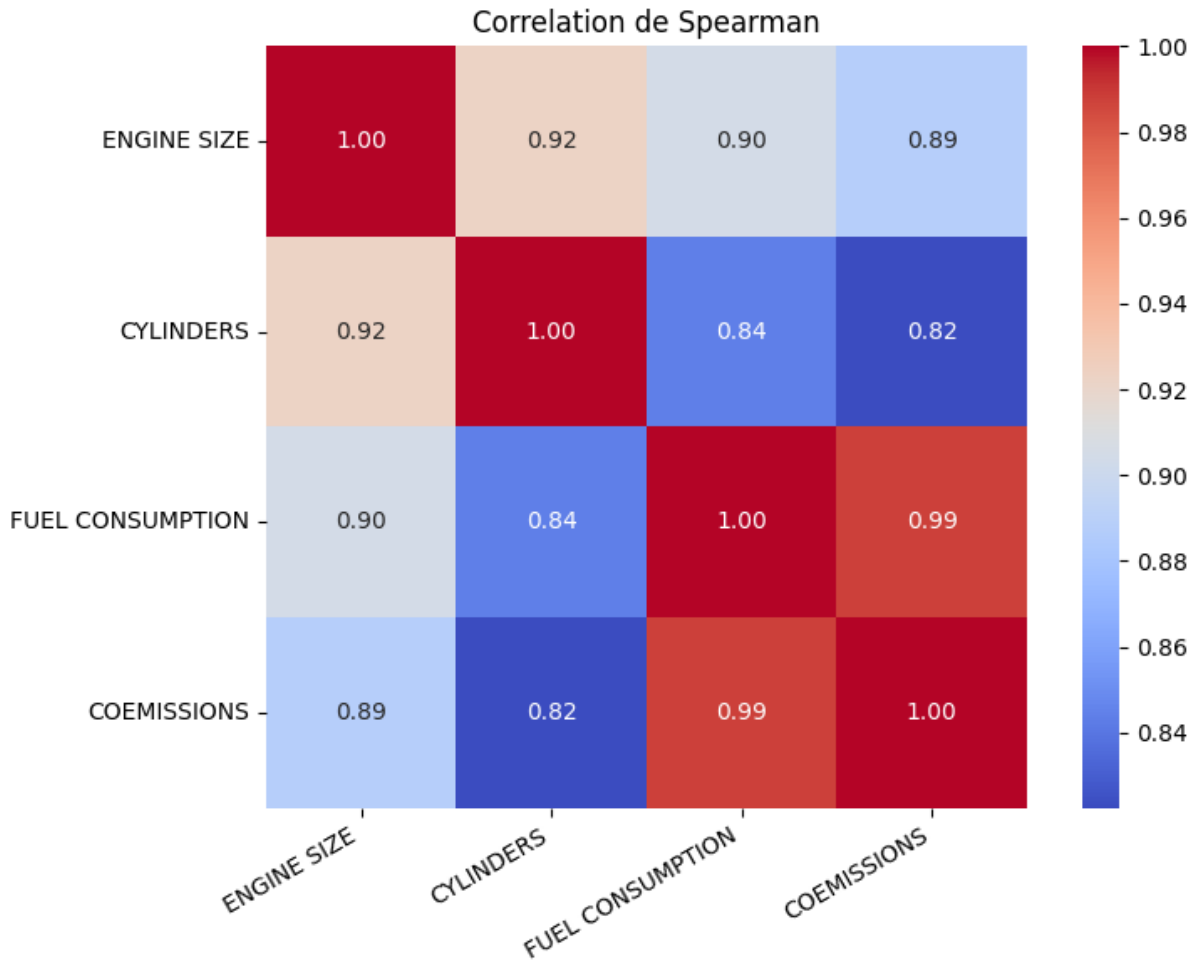


FIGURE 2 – Matrice de corrélation (Spearman)

## 5 Architecture du modèle

### 5.1 Topologie

L'architecture suit le TP de référence :

- deux couches LSTM de 12 unités (la première avec `return_sequences=True`) ;
- une couche dense de 32 unités (ReLU) ;
- une sortie linéaire unique pour la régression.

Compilation : optimiseur `adamax`, perte MAE, métrique `mae`.

### 5.2 Raisons du choix

La topologie est volontairement très petite ( 2 300 paramètres) pour permettre un déploiement embarqué tout en capturant des interactions non linéaires simples entre variables.

## 6 Entraînement

Le modèle est entraîné avec les paramètres suivants : `epochs=300`, `batch_size=32`, `validation_split=0.2`.

La courbe de perte (MAE) est sauvegardée : `figs/tp3_loss.png`.

## 7 Évaluation et résultats

Après entraînement, le notebook calcule : MAE, RMSE,  $R^2$ , et exporte les résultats dans `figs/tp3_results.csv`. Les figures suivantes sont produites :

*Fichier non trouvé : figs/tp3\_residuals.png*

La figure sera affichée automatiquement dès que le notebook aura généré ce fichier.

FIGURE 5 – Analyse des résidus

### 7.1 Métriques (placeholders)

Si le notebook a déjà produit `figs/tp3_results.csv`, vous pouvez remplir les valeurs ci-dessous. Sinon, elles seront

| oprule Métrique | Valeur      | Unité   |
|-----------------|-------------|---------|
| MAE             | à compléter | L/100km |
| RMSE            | à compléter | L/100km |
| $R^2$           | à compléter | –       |

TABLE 1 – Synthèse des performances sur l'ensemble de test.

## 8 Analyse critique

### 8.1 Points forts

- chaîne simple et reproductible ;
- topologie suffisamment compacte pour TinyML ;
- standardisation correcte (fit sur train uniquement) ;
- figures et résultats exportés pour diagnostic rapide.

### 8.2 Limites

- usage d'une fenêtre courte (`timesteps=3`) - limite la capture de dépendances longues ;
- pas d'early stopping ni de calibration fine des hyperparamètres ;
- modèle simpliste pour des dynamiques non linéaires complexes.

### 8.3 Pistes d'amélioration

- augmenter la fenêtre temporelle ou empiler davantage de couches ;
- ajouter des features (interactions, polynômes, transformations log) ;
- utiliser EarlyStopping et une recherche d'hyperparamètres ;
- comparer avec GRU / TCN et modèles plus modernes si la contrainte de taille le permet.

## 9 TinyML — Export et déploiement

Le notebook propose un export optionnel via eloquent-tensorflow qui convertit le modèle Keras en code C embarquable (fichier model\_tinyml.h). Cette conversion est conditionnelle à la présence de la librairie.

## 10 Conclusion

Le notebook fournit une implémentation compacte et reproductible pour la prédiction de la consommation de carburant. Son principal atout est la portabilité vers des systèmes embarqués ; ses limites tiennent surtout à la simplicité du modèle et à l'absence d'une recherche d'hyperparamètres plus systématique.

## A Correspondance avec le notebook

Les sections du rapport correspondent aux blocs du notebook dans l'ordre suivant :

1. imports et configuration ;
2. chargement du CSV et nettoyage ;
3. EDA (pairplot, heatmap) ;
4. préparation (scaler, reshape) ;
5. définition du modèle (build\_lstm) ;
6. entraînement (history) ;
7. évaluation (prédictions, résidus) ;
8. export TinyML (optionnel).

## B Extraits de code essentiels

```
# Construction du modèle (extrait)
def build_lstm(input_shape=(3, 1)):
    model = tf.keras.Sequential([
        layers.Input(shape=input_shape),
        layers.LSTM(12, return_sequences=True),
        layers.LSTM(12),
        layers.Dense(32, activation="relu"),
        layers.Dense(1, activation="linear"),
    ])
    model.compile(optimizer="adamax", loss="mae", metrics=["mae"])
    return model

history = model.fit(X_train, y_train, validation_split=0.2, epochs=300, batch_size=32, verbose=0)

y_pred = model.predict(X_test, verbose=0)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```